

The Google Matrix Page Ranking Method

CLA365 TECHNICAL REPORT

Jay Lebakken

Ben Crotteau

Xander Hall

September 30, 2026

Abstract

In this report we will explore the properties of the Google Matrix Page Rank method, and analyze how different probabilistic values impact the outcomes of google page ranking. Additionally, We also supply r code for creating a Google matrix from an adjacency Matrix A, and do an in-depth analysis of a specific example.

1 Google Matrix

1.1 Overview

The Google Matrix is a method of page ranking that ranks pages based on "Importance". Importance is heuristically defined in this problem as nodes that are linked to by nodes that have high importance. While a naive approach would be to rank pages by their in-degree rank (number of in-going vertices), the google matrix takes into account the importance of nodes that point towards a given vertex. To construct this, the google matrix employs a random walk on the adjacency matrix A, where at any time state, the walker at point $A_{i,j}$ has q probability of going to any vertex in A, and $1 - q$ probability of going to nodes with edges to A_i . The probability of being at node j p_j can be found with the equation

$$p_j = \sum_i \left(\frac{qP_i}{n} + (1 - q) \frac{P_i}{n_i} A_{i,j} \right)$$

1.2 Implementation

```
1 GoogleMatrix <- function(A,q){
2   ##parameters:
3   ## an adjacency matrix A representing a network
4   ## a probabilistic value q, representing the chance that a random walk
5   ## goes to any page in A
6   B <- diag(1,nrow(A))
7   for( i in 1:length(A[,1])){
8     for( j in 1:length(A[,1])){
9       B[i,j] <- (q/length(A[,1])) + A[j,i]*(1-q)/sum(A[j,])
10    }
11  }
12  return(B)
13 }
```

This is the code to create a Google matrix, given an nxn matrix A. In this report we will be doing an in-depth analysis of a specific example (figure 1) and its corresponding Google Matrix (figure 2). In figure 1 we can see a connected network of webpages (nodes) and links (directed edges). Our adjacency matrix A models this diagram.

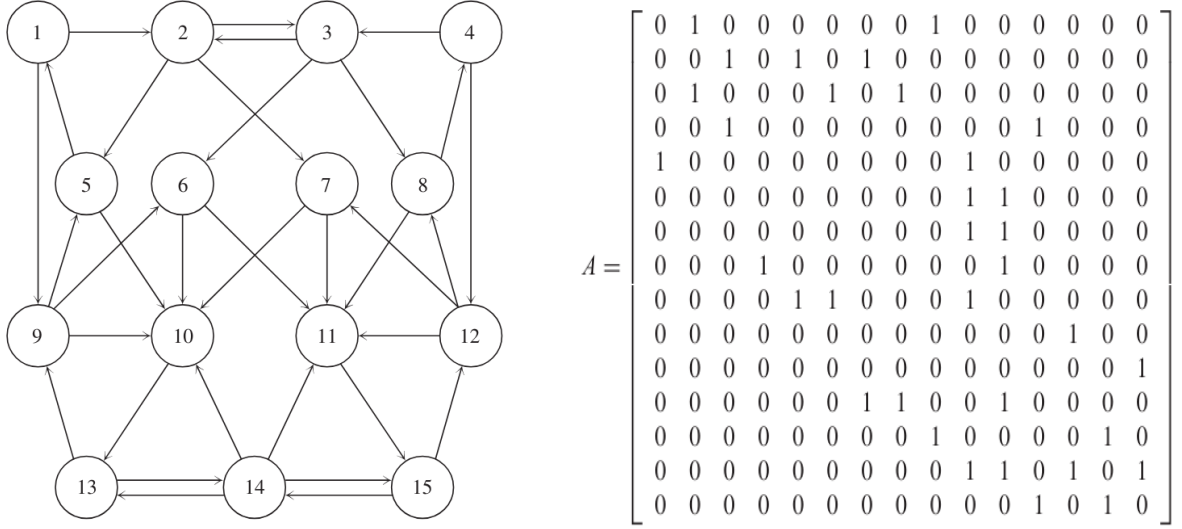


Figure 1: Our example network of websites and pages and its corresponding adjacency matrix

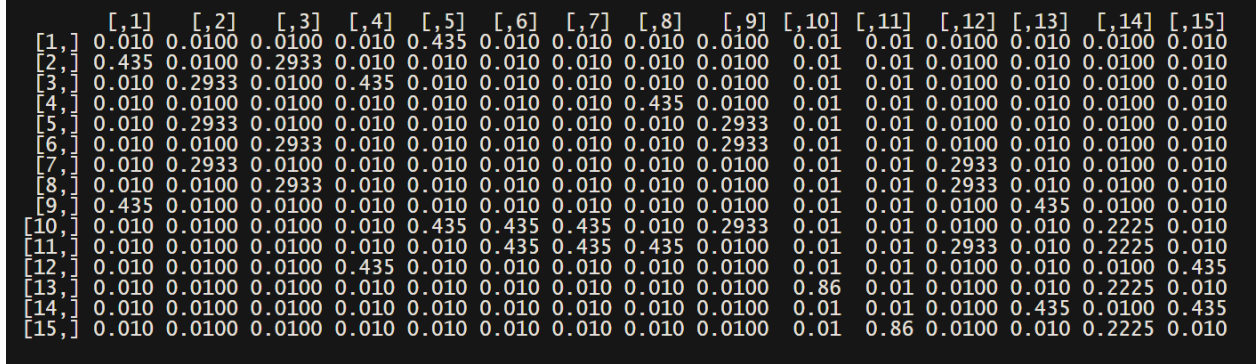


Figure 2: Our Google Matrix for $q=0.15$

2 Analysing different values of Q

Let's first begin by analysis $q=0.15$. This implies the probability of a random page change is quite low. The principal eigenvector (eigenvector of dominant eigenvalue 1) is:

```

1 gexample <- GoogleMatrix(example)
2 eigen <- eigen(gexample)
3 principal <- eigen$vectors[,1] #Our principal eigenvector
4 principal2 <- prin/sum(prin) #Scaling so probabilities add to 1
5 principal2

```

$$q_{0.15} = \begin{pmatrix} 0.0268 \\ 0.0299 \\ 0.0299 \\ 0.0268 \\ 0.0396 \\ 0.0396 \\ 0.0396 \\ 0.0396 \\ 0.0746 \\ 0.1063 \\ 0.1063 \\ 0.0746 \\ 0.1251 \\ 0.1163 \\ 0.125 \end{pmatrix} \quad q_{0.5} = \begin{pmatrix} 0.0467 \\ 0.0540 \\ 0.0540 \\ 0.0467 \\ 0.0536 \\ 0.0536 \\ 0.0536 \\ 0.0536 \\ 0.0676 \\ 0.0946 \\ 0.0946 \\ 0.0676 \\ 0.0905 \\ 0.0786 \\ 0.0905 \end{pmatrix}$$

Increasing q is increasing the probability of a random jump to a different website. Thus, the directed edges in our GoogleMatrix become less influential since there is such a high chance of moving to a random site regardless. This can be seen in our principal eigenvector for $q=0.15$ and $q=0.5$. For $q=0.15$ the highest page rank is node 13 and 15 with 0.1251, and our smallest is node 1 and 4 with 0.0268. The difference between the largest and smallest page rank is 0.0983. For $q=0.5$, the largest page rank is now nodes 10 and 11 at 0.0946, and the smallest is node 1 and 4 with 0.0467. The difference is now only 0.0479, almost half of the previous one. The higher random jump probability brings the distribution much closer, since the nodes with high page ranks are made less important following the lower value of a directed edge, and nodes with few directed edges are more often visited due to the random jumps. Why does the largest page rank change for $q=0.5$?

3 Promoting Pages

Another thing to consider is pages paying others to more prominently display its links in order to get more traffic and improve its page rank. To model this, we replaced a 1 in the adjacency matrix with a 2 for the ij^{th} entry in A if site i promotes the link of site j .

As an example, suppose that page 7 paid page 2 and page 12 to put its link in a more visible spot on their sites. In A , the 1 in $A_{2,7}$ and $A_{12,7}$ was replaced by a 2, and the same process as before was used to analyze page rank. After finding the dominant eigenvalue and scaling so its entries sum to 1, the new page rank is:

$$p_{new} = \begin{pmatrix} 0.026 \\ 0.028 \\ 0.026 \\ 0.024 \\ 0.038 \\ 0.039 \\ 0.053 \\ 0.033 \\ 0.076 \\ 0.112 \\ 0.103 \\ 0.072 \\ 0.130 \\ 0.117 \\ 0.123 \end{pmatrix}$$

The difference between the new page rank and the original is:

$$p_{difference} = p_{new} - p = \begin{pmatrix} -0.00083 \\ -0.00138 \\ -0.00363 \\ -0.00288 \\ -0.00195 \\ -0.00057 \\ 0.01325 \\ -0.00679 \\ 0.00162 \\ 0.00523 \\ -0.00305 \\ -0.00224 \\ 0.00465 \\ 0.00096 \\ -0.00239 \end{pmatrix}$$

We can see that the page rank for site 7 improves the most. This makes sense as they paid to promote their link on sites 2 and 12. Interestingly, the page rank for sites 2 and 12 decrease after they promote the link to site 7. Analyzing the adjacency matrix, site 7 does not have an edge from itself to site 2 or site 12, but sites 2 and 12 have an edge from themselves to site 7. As site 7 grows in importance, it makes sense that the page ranking for sites 2 and 12 decrease. The other sites that have an increased page ranking are sites 9, 10, 13, and 14. There is an edge directed from site 7 to site 10 in the adjacency matrix, from site 10 to site 13, site 13 to sites 9 and 14, site 9 to site 10, and site 14 to sites 10 and 13. Thus, there is an interdependence between these four sites, and since site 7 depends on one of the four (site 10), it makes sense that there page ranking all increases as site 7's increases.

4 Removing node 10

To see the impacts of node 10 (the node with most edges), let's see what happens when we remove it!

Doing the same breakdown as for the previous two sections (with $p=0.15$) our page rank becomes:

$$p_{-10} = \begin{array}{l} 0.0471 \\ 0.0409 \\ 0.0359 \\ 0.0321 \\ 0.0428 \\ 0.04139 \\ 0.0517 \\ 0.05025 \\ 0.0482 \\ 0.1710 \\ 0.1036 \\ 0.0412 \\ 0.1075 \\ 0.1865 \end{array}$$

Since 10 has been removed, certain nodes become far more important. Node 15 has risen from 0.1251 to 0.1865 and node 11 from 0.1251 to 0.1710. Since 10, one of our more important nodes previously, has been removed, other central nodes have to take up the slack. Interestingly, node 13 has lost almost all of its importance! It went from our most important, to the second lowest. This is likely due to the fact that 10 has many sites linking to it, but only linked to node 13. 10 this acted as an intermediary for other nodes to reach 13. With 10 gone, 13 has thus lost almost all of its importance, while only lose one directed edge. This highlights how important the whole system is, and not simply the individual directed edges are.

5 Proof of Stochastic Properties of the Google Matrix

We know that the Google matrix will always be stochastic. Stochastic matrices represent probabilities at a certain state in each column, so every column will sum to one. To show this, we will take an arbitrary column of G , our google matrix, which we will call c . We will say that this column of G is built from a row of the adjacency matrix A , with α ones (representing websites). Then, our column of G will have $\sum_{i=0}^n \frac{q}{n} = n(\frac{q}{n}) = q$ distributed evenly across all n entries, representing the chance that the current page was reached from a random click. Then, we will have α entries with the additional probability that they were linked to by one of the α sites from the corresponding row of A . This becomes $\alpha(\frac{q-1}{\alpha}) = (1-q)$. To find the total sum of any column c , we can follow the equation

$$\begin{aligned} sum(c) &= \sum_{i=1}^n \frac{q}{n} + \sum_{i=1}^{\alpha} \frac{1-q}{\alpha} \\ &= n(\frac{q}{n}) + \alpha(\frac{1-q}{\alpha}) \\ &= q + 1 - q \\ sum(c) &= 1 \end{aligned}$$

Thus the Google matrix is stochastic

6 Conclusion

In conclusion, the google matrix provides a comprehensive and effective measure of centrality, although it is not without it's flaws. By analyzing the dominant eigenvector of the (stochastic) Google Matrix for a given network, we can easily extract the sites with the highest "Importance". Pages considered important are linked to by many sites also considered important, and pages are penalized for being linked to by many unimportant sites. As we analyzed in this report, one way that websites can artificially raise their page rank is by paying for high ranking sites to link to their page.

References

- [1] Sauer, Tim. Numerical analysis / Timothy Sauer. – 2nd ed. p. cm. Includes bibliographical references and index. ISBN-13: 978-0-321-78367-7 ISBN-10: 0-321-78367-0

7 Appendix

```
1
2
3 G <- function(A,q=0.15){
4   B <- diag(1,nrow(A))
5   for( i in 1:length(A[,])){
6     for( j in 1:length(A[,1])){
7       B[i,j] <- (q/length(A[,1])) + A[j,i]*(1-q)/sum(A[j,])
8     }
9   }
10  }
11  }
12  return(B)
13 }
14
15
16 options(digits=4)
17 example <- spMatrix(15,15,i=c
18   (5,1,3,2,4,8,2,9,3,9,2,12,3,12,1,13,5,6,7,9,14,6,7,8,12,14,4,15,10,14,13,15,11,14),j=c
19   (1,2,2,3,3,4,5,5,6,6,7,7,8,8,9,9,10,10,10,10,10,11,11,11,11,11,12,12,13,13,14,14,15,15),
20   x=c(rep(1,34)))
21 #example
22
23 #sum(example[,1])
24
25 gexample <- G(example)
26
27 gexample
28
29 #sum(gexample[,1])
30
31 eigen <- eigen(gexample)
32 #eigen
33 #max(abs(eigen$values))
34
35 prin <- eigen$vectors[,1]
36 #prin
37 prin2 <- prin/sum(prin)
38 #sum(prin2)
39 #prin2
40
41 #promoting page 7 by putting a 2 in 2,7 and 12,7
42 newA <- A
43 newA[2,7] <- 2
44 newA[12,7] <- 2
45
46 #creating the Google matrix from the promoting pages section and finding the dominant
47   eigenvector
48 pageBoost <- G(newA)
49 newPageRank <- PI(pageBoost)$vec/(sum(PI(pageBoost)$vec))
50 newPageRank
51
52 #New matrix with node 10 removed
53 example <- spMatrix(14,14,i=c
54   (5,1,3,2,4,8,2,9,3,9,2,11,3,11,1,12,6,7,8,11,13,4,14,13,12,14,10,13),j=c
55   (1,2,2,3,3,4,5,5,6,6,7,7,8,8,9,9,10,10,10,10,10,11,11,12,13,13,14,14),x=c(rep(1,28)))
56 #example
57
58 #sum(example[,1])
```

```
53 |
54 | gexample <- G(example)
55 |
56 | #gexample
57 |
58 | sum(gexample[,1])
59 |
60 | eigen <- eigen(gexample)
61 | #eigen
62 | #max(abs(eigen$values))
63 |
64 | prin <- eigen$vectors[,1]
65 | #prin
66 | prin2 <- prin/sum(prin)
67 | sum(prin2)
68 | prin2
```